ADAPTIVE PRICING AGENT

AI-Driven Dynamic Fare Pricing for Independent Taxi Drivers

DATA SCIENCE PROJECT

PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE **DEGREE OF BACHELOR OF ENGINEERING** in **Computer Science and Engineering** OF THE ANNA UNIVERSITY

OCTOBER 2026

PROJECT WORK

BATCH 2024 – 2028

Submitted by

XXXXXX-REGNUMBER

Under the Guidance of

Dr.M.Suriya, M.E.,Ph.D

Associate Professor , CSE

Computer Science and Engineering

SRI ESHWAR COLLEGE OF ENGINEERING

(An Autonomous Institution – Affiliated to Anna University)

COIMBATORE – 641 202



ADAPTIVE PRICING AGENT

AI-Driven Dynamic Fare Pricing for Independent Taxi Drivers

DATA SCIENCE PROJECT REPORT

Submitted by

XXXXXX **[ROLLNO]**

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

SRI ESHWAR COLLEGE OF ENGINEERING

(AN AUTONOMOUS INSTITUTION)

COIMBATORE – 641 202

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report **"ADAPTIVE PRICING AGENT"** is the bonafide work of

XXXXXX **[ROLLNO]**

Who carried out the Data Science Project work under my supervision

…… ……

SIGNATURE **SIGNATURE**

Dr.R.Subha M.E.,Ph.D
Professor &
Head of the Department,
Department of Computer Science
and Engineering,
Sri Eshwar College of Engineering,
Coimbatore-641202.

Dr.M.Suriya, M.E., Ph.D.,
Associate Professor,
Department of Computer Science
and Engineering,
Sri Eshwar College of Engineering,
Coimbatore-641202.

Submitted for the End Semester Project examination **U23AD491 Data Science** – Project work viva-voice held on ……

…… ……

(Internal Examiner) **(External Examiner)**

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**i**
	LIST OF TABLES	**ii**
	LIST OF FIGURES	**iii**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 OBJECTIVES	1
	1.3 SCOPE	2
	1.4 TECHNOLOGY STACK	2
2	**SYSTEM ANALYSIS AND DESIGN**	**3**
	2.1 LITERATURE SURVEY	3
	2.2 EXISTING SYSTEM	4
	2.3 PROBLEM STATEMENT	4
	2.4 REQUIREMENTS	4
3	**PROPOSED SYSTEM**	**6**
	3.1 OVERVIEW	6
	3.2 BLOCK DIAGRAM	6
	3.3 MAJOR MODULES	7
	3.4 API DESIGN	8
4	**PROJECT DESCRIPTION**	**9**
	4.1 DATASET COLLECTION AND DESCRIPTION	9
	4.2 METHODOLOGY	10
	4.3 IMPLEMENTATION	12
5	**RESULTS AND INFERENCES**	**15**
6	**CONCLUSION & FUTURE SCOPE**	**28**
	REFERENCES	**30**

ABSTRACT

The **Adaptive Pricing Agent** is an AI-driven dynamic pricing system that helps independent taxi drivers — drivers who work for themselves rather than under brands such as Red Taxi, Blue Taxi or Rapido — quote a fare that customers will choose over the branded operators while still protecting the driver's profit. Branded fleets and ride-hailing apps price trips with data and surge algorithms; an independent driver usually quotes by guesswork, and either loses the ride by pricing too high or loses money by pricing too low, particularly when the return journey is empty.

The system gathers live signals for each trip: driving distance and duration from OpenStreetMap (Nominatim geocoding and OSRM routing), current weather and rain intensity from OpenWeather, the time of day, the driver-reported availability of taxis nearby and the fares quoted by competitors. A transparent rule-based fare engine converts these signals into a cost floor (fuel including the empty return, tolls and a minimum profit), a demand-pressure score between 0 and 1, and an undercut that moves linearly from 12% to 3% below the cheapest competitor as demand rises. Fares are charm-priced to end in 9 and offered as three options — quick win, recommended and maximum earnings.

A large language model (Groq gpt-oss-120b in JSON mode) reviews the recommendation, may fine-tune it only inside a guaranteed-safe range, and writes a WhatsApp-ready customer message in English or Tamil. Validators reject malformed or wrong-language responses and fall back to deterministic templates. Booked and missed outcomes recorded by the driver are stored in SQLite and fed back into the engine, shifting the undercut between −2 and +3 percentage points according to the driver's own win rate. The workflow is orchestrated as a LangGraph state graph with a parallel branch and a conditional fallback; signal and pricing tools are exposed through a Model Context Protocol (MCP) server; a FastAPI backend streams results to a responsive web interface.

On the reference trip from Coimbatore Junction to Coimbatore International Airport (11.1 km), with rivals quoting ₹600 and ₹580, rain and very few taxis nearby, the agent recommends **₹549** — ₹31 below the cheapest rival while the driver keeps about ₹398. Across seven demand scenarios the recommended fare ranged from ₹509 to ₹559, always below the rival and above the ₹309 cost floor. In an evaluation of 20 live LLM calls, 19 responses passed validation and 19 of 19 accepted fares stayed inside the safe range. End-to-end latency was reduced from 20.8 s to 6.0 s, with the fare visible after 3.4 s, and all 24 automated unit tests pass.

LIST OF TABLES

Table No.	Title
Table 1	Project technology stack
Table 2	Project modules and their responsibilities
Table 3	Representative REST API endpoints
Table 4	Data sources and fields used
Table 5	Structure of a stored quote record
Table 6	Market-estimate rates used when no rival fare is given
Table 7	Demand-pressure weights
Table 8	Main source files
Table 9	Quote for the reference trip
Table 10	Recommended fare under different demand conditions
Table 11	Results on real Coimbatore routes
Table 12	Effect of the driver's win rate on the recommendation
Table 13	LLM guardrail evaluation (n = 20)
Table 14	Measured end-to-end latency

LIST OF FIGURES

Figure No.	Title
Figure 1	Adaptive Pricing Agent system architecture and data flow
Figure 2	LangGraph state graph executed for each taxi quote
Figure 3	Pricing methodology pipeline
Figure 4	Undercut as a function of demand pressure, with the evaluated scenarios
Figure 5	Recommended fare across demand scenarios versus the rival fare and cost floor
Figure 6	Market estimate, recommended fare and cost floor on real routes
Figure 7	Recommended fare as a function of the driver's win rate
Figure 8	LLM responses passing validation and response-time distribution
Figure 9	End-to-end latency before and after optimisation
Figure 10	Application home screen with trip form and the winning fare
Figure 11	Search-as-you-type place suggestions biased to Coimbatore
Figure 12	Live progress driven by the streamed graph nodes
Figure 13	Winning fare card with three options and outcome buttons
Figure 14	Comparison with rival fares and the demand-pressure gauge
Figure 15	Trip details and route map
Figure 16	Factors driving the price, including the learned win-rate factor
Figure 17	WhatsApp-ready customer message and money breakdown
Figure 18	AI explanation of the recommended price
Figure 19	Driver track record: win rate, earnings and recent quotes
Figure 20	Phone layout with a Tamil customer message
Figure 21	Streaming API output — one JSON line per completed graph node
Figure 22	Automated test suite output — all 24 tests passed
Figure 23	Interactive API documentation generated by FastAPI
Figure 24	Retail pricing module from the first project phase

1. INTRODUCTION

1.1 Background

Urban mobility in Indian tier-2 cities such as Coimbatore is served by three kinds of providers: branded taxi fleets (for example Red Taxi and Blue Taxi), ride-hailing and bike-taxi platforms (Rapido, Ola, Uber, Namma Yatri), and a large number of independent drivers who own or rent a single car and find customers at railway stations, bus stands, the airport and through word of mouth. Branded fleets and platforms price every trip with software: distance and time based tariffs, surge multipliers when demand exceeds supply, and continuous experimentation with their own data.

An independent driver has none of these tools. When a passenger at Coimbatore Junction asks for a fare to the airport, the driver has to decide in a few seconds — usually from memory — without knowing the exact road distance, how many competing taxis are nearby, whether the rain will push branded fares up, or how much the empty drive back will cost. Quoting too high loses the passenger to a branded taxi; quoting too low wins the ride but can leave the driver with little or no profit once fuel for the return journey is counted.

The **Adaptive Pricing Agent** addresses this gap. It treats pricing as a data problem: it collects live trip signals, computes the driver's real cost, reads the competitive market, estimates demand pressure, and recommends a fare that is attractive enough to win the customer while staying profitable. A large language model then explains the decision and writes a short customer message the driver can send immediately. The project began as a retail price-recommendation prototype for market vendors (perishable goods priced by stock, expiry, competitor price and weather) and was extended into the taxi domain, which is the main focus of this report.

1.2 Objectives

- Collect live, trip-level signals — route distance and duration, weather and rain intensity, time of day, local taxi availability and competitor fares — from open data services and the driver.
- Compute a transparent **cost floor** for every trip that includes fuel for the trip and the empty return, tolls or parking, and the driver's minimum acceptable profit.
- Quantify **demand pressure** on a 0–1 scale and use it to decide how far below the cheapest competitor the driver must price to win the ride.

- Recommend a charm-priced fare and three selectable options (quick win, recommended, maximum earnings) that are always profitable and, where possible, cheaper than every branded rival.
- Use an LLM to review the recommendation within a guaranteed-safe range and generate a customer pitch in English or Tamil, with validation and deterministic fallbacks.
- Learn from the driver's booked and missed quotes and adapt future pricing to the driver's own win rate.
- Orchestrate the pipeline as an explicit LangGraph state graph, expose tools through the Model Context Protocol, and deliver results through a fast, streaming web interface.

1.3 Scope

The scope covers signal acquisition, data validation and caching, the rule-based pricing model, the outcome-based learning loop, LLM-based review and message generation, orchestration, a REST and streaming API, persistence, a responsive driver-facing web application, automated tests and a quantitative evaluation. The system is designed for a single independent driver operating in and around Coimbatore; distances and default market rates are calibrated to that region but the method is location-independent. Fare recommendations are decision support — the driver always sets the final price.

1.4 Technology Stack

Table 1: Project technology stack

Layer	Technologies
Frontend	HTML5, CSS3 (glassmorphism, animations), vanilla JavaScript, Leaflet maps, canvas-confetti, Google Fonts
Backend	Python 3.12, FastAPI, Uvicorn, Pydantic, StreamingResponse (NDJSON)
Agent orchestration	LangGraph state graph (parallel branch, conditional fallback, RunnableConfig injection)
Tool protocol	Model Context Protocol (MCP) — FastMCP server over stdio, persistent client session
AI model	Groq API — openai/gpt-oss-120b (JSON mode, low reasoning effort) with gpt-oss-20b and Qwen fallbacks
Data sources	OpenStreetMap Nominatim and Photon (geocoding, place search), OSRM (routing), OpenWeather (weather)
Database	SQLite — quote history with outcomes, 30-day cache of map lookups
Testing	Python unittest (24 tests), headless Chrome via DevTools protocol for UI verification

2. SYSTEM ANALYSIS AND DESIGN

2.1 Literature Survey

Dynamic pricing. Gallego and van Ryzin [1] showed that when demand is stochastic and capacity is perishable, the revenue-maximising price should change with the remaining time and inventory — a taxi seat that leaves empty, like an unsold hotel room, has zero salvage value. Den Boer [2] surveys the later work on pricing while learning the demand curve, where the seller adjusts prices from observed customer responses; the outcome-based learning loop in this project is a simple, interpretable instance of that idea.

Surge pricing in ride-hailing. Cachon, Daniels and Lobel [3] model platforms whose drivers choose when to work and show that price that responds to demand improves both service levels and driver earnings. Castillo, Knoepfle and Weyl [4] explain why, without surge pricing, scarce drivers waste time travelling to distant pickups (the "wild goose chase"). Chen, Mislove and Wilson [5] measured Uber's surge behaviour empirically, and Cohen et al. [6] used surge-price variation to estimate customer willingness to pay. Together these works establish that rain, peak hours and low driver supply measurably raise riders' willingness to pay — the signals this project turns into a demand-pressure score. Our design inverts the platform view: instead of raising price when demand is high, the independent driver uses high demand to **undercut the branded surge by a smaller margin** and keep more profit.

Price perception. Thomas and Morwitz [7] demonstrated the left-digit effect: prices ending just below a round number (₹549 rather than ₹550) are perceived as noticeably cheaper. The fare engine therefore charm-prices every recommendation to end in 9, rounding down when competing and rounding up when protecting the cost floor.

LLM agents and tool use. Yao et al. [8] introduced ReAct, in which language models interleave reasoning with calls to external tools. The Model Context Protocol [9] standardises how such tools are described and invoked, and LangGraph [10] represents an agent workflow as an explicit state graph with typed state, parallel branches and conditional edges. This project uses all three ideas but keeps the numerical decision in a deterministic engine; the LLM only reviews within bounds and writes language, following the general principle of constraining model output that has financial consequences.

Open geodata. OpenStreetMap [11] provides crowd-sourced road data, and OSRM [12] computes fast driving routes on it, which makes accurate distance and duration available without a commercial maps licence.

2.2 Existing System

Today an independent driver prices a trip using one of the following approaches, none of which uses data:

- **Fixed personal tariff** — a per-kilometre rate remembered from experience, which ignores rain, peak hours, scarcity and what competitors are charging at that moment.
- **Negotiation at the stand** — the price depends on bargaining, which takes time and often ends below cost when the driver fears losing the ride.
- **Checking a rival app manually** — the driver may open a ride-hailing app to see its quote, but must then decide alone how much cheaper to be and whether that is still profitable.
- **Joining an aggregator** — the driver gains demand-based pricing but gives up a commission of each fare and control over the price.

Branded operators, in contrast, have complete pricing systems, but they are closed and optimise for the platform. There is no tool that gives an independent driver the same pricing intelligence while keeping the driver in control.

2.3 Problem Statement

Design and build an AI-assisted pricing agent that, for any trip requested from an independent taxi driver, combines live route, weather, time and competition signals with the driver's real operating costs to recommend a fare that is lower than the cheapest branded alternative whenever this is profitable, never below the driver's cost floor, explained in plain language, packaged as a ready-to-send customer message, and continuously adapted to the driver's own booking outcomes.

2.4 Functional Requirements

- Accept pickup, drop, trip time, vehicle type, competitor quotes, taxi availability, weather (live or driver-reported) and the driver's cost parameters.
- Suggest places while typing and support the device's current location as pickup.
- Compute route distance, travel time with traffic, weather, cost floor, demand pressure and three fare options.
- Generate reasoning and a customer message in English or Tamil, signed with the driver's name and phone number.
- Record whether each quote was booked or missed and show the driver's win rate and earnings.
- Expose all functions through a documented REST API, including a streaming endpoint.

2.5 Non-Functional Requirements

- **Safety of advice** — no recommendation, including AI-adjusted ones, may fall below the cost floor.
- **Responsiveness** — the fare should appear within a few seconds; slower AI text may follow.
- **Robustness** — failures of weather, routing or the LLM must degrade gracefully instead of stopping the quote.
- **Usability** — the interface must work on a phone, in bright and dark conditions, and in the driver's language.

3. PROPOSED SYSTEM

3.1 Overview

The proposed system follows a layered, agent-based architecture (Figure 1). The **presentation layer** is a single-page web application used by the driver. The **service layer** is a FastAPI server that validates requests, invokes the agent and streams each intermediate result back as a line of JSON. The **orchestration layer** is a LangGraph state graph that decides which step runs next and what happens when a step fails. The **tool layer** is an MCP server named taxi-signals that exposes three tools — route lookup, weather lookup and fare computation — and talks to the external data providers. The **intelligence layer** is the Groq-hosted LLM, and the **persistence layer** is a SQLite database that stores quote history, outcomes and a cache of map lookups.

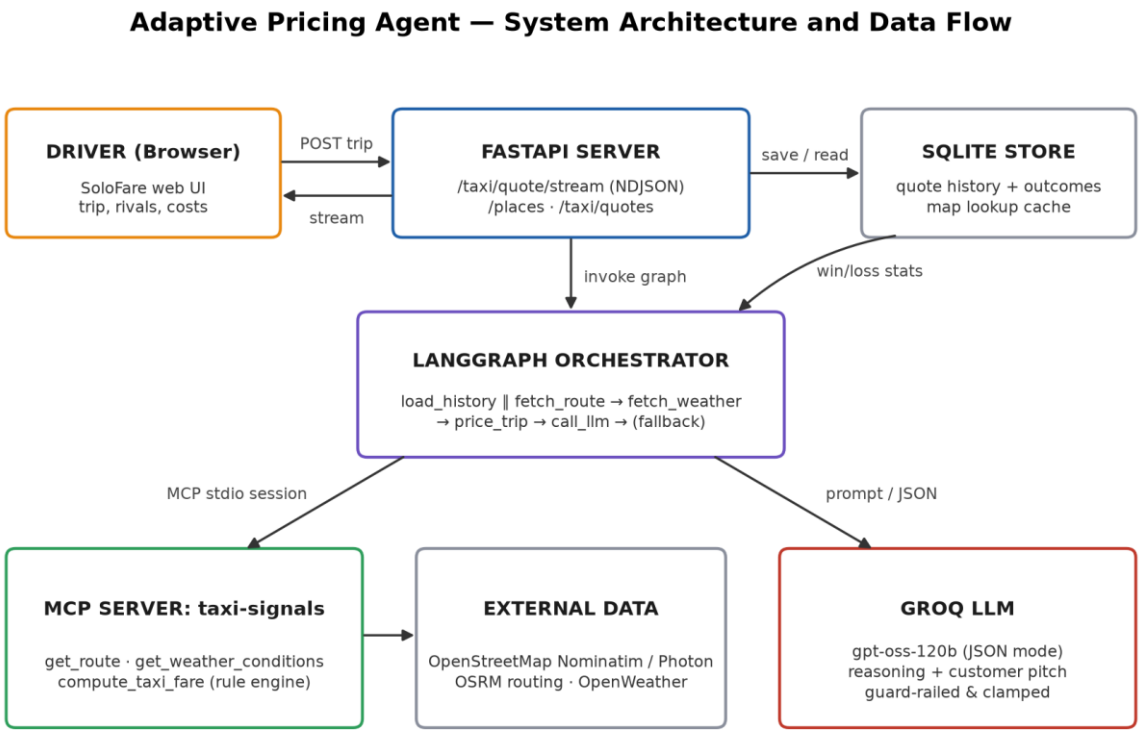


Figure 1: Adaptive Pricing Agent system architecture and data flow

3.2 Block Diagram

Figure 2 shows the LangGraph state graph executed for every quote. Loading the driver's history runs in parallel with the route lookup; the weather lookup needs the pickup coordinates and therefore follows the route. Both branches join at the pricing step. The LLM step is followed by a conditional edge: valid advice ends the graph, while any error routes to a deterministic fallback so that the driver always receives a complete answer.

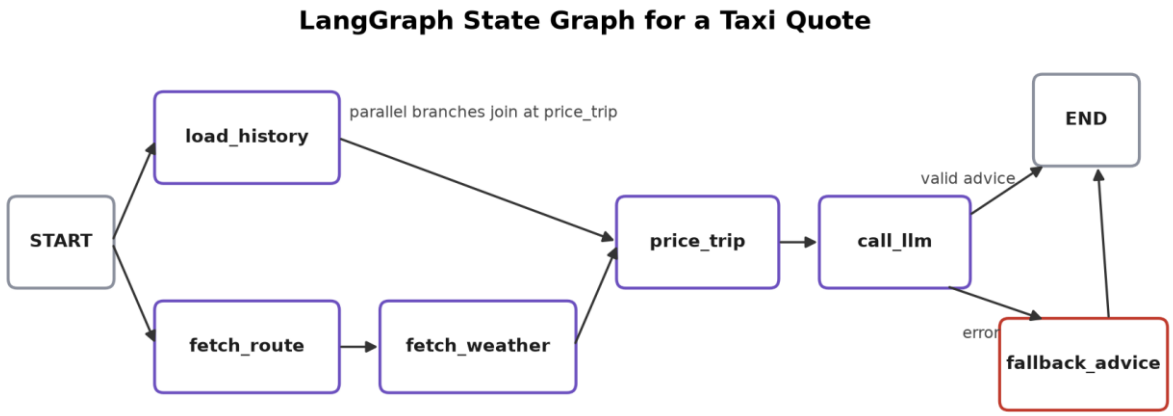


Figure 2: LangGraph state graph executed for each taxi quote

Figure 3 summarises the pricing methodology as a data-processing pipeline, from raw trip inputs to the feedback that closes the learning loop.

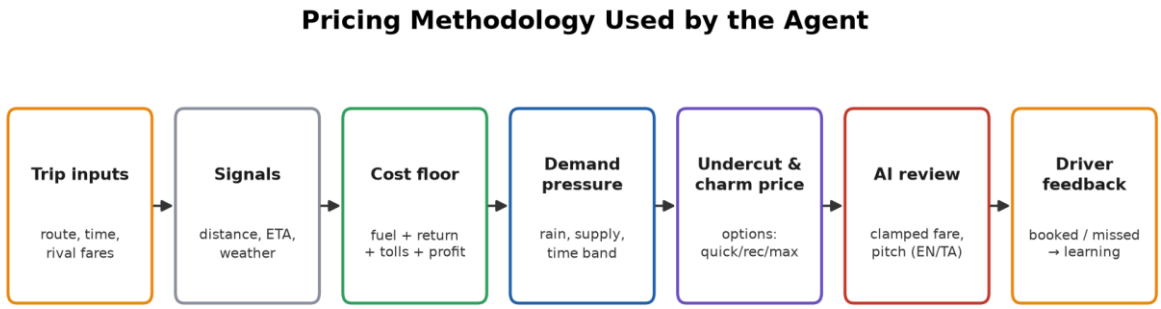


Figure 3: Pricing methodology pipeline

3.3 Major Modules

Table 2: Project modules and their responsibilities

Module	Responsibility
Signal collection	Geocode places, compute OSRM driving routes with a straight-line fallback, read current weather and rain intensity.
Fare engine	Cost floor, market anchor, demand pressure, undercut, charm pricing, three options, traffic-aware ETA, learned adjustment.
MCP tool server	Expose get_route, get_weather_conditions and compute_taxi_fare as typed MCP tools over stdio.
Agent orchestrator	LangGraph graph with typed TaxiState, parallel branch, join and conditional fallback.
AI strategist	Build the prompt, call Groq in JSON mode, validate and clamp the result, produce templated pitches.
Persistence	SQLite quote history, booked/missed outcomes, win-rate statistics and 30-day lookup cache.
API service	Validated REST endpoints, NDJSON streaming, place search, history and outcome recording.

Module	Responsibility
Web application	Driver UI with live progress, fare options, charts, map, WhatsApp sharing, history and Tamil support.

3.4 API Design

Table 3: Representative REST API endpoints

Endpoint	Purpose
GET /	Serves the taxi fare web application
POST /taxi/quote/stream	Runs the agent and streams one JSON line per completed graph node
POST /taxi/quote	Runs the agent and returns the complete quote in one response
POST /taxi/quotes/{id}/outcome	Records whether a quote was booked or missed and the fare offered
GET /taxi/quotes	Returns recent quotes and win-rate statistics
GET /places?q=	Search-as-you-type place suggestions biased to Coimbatore
GET /places/reverse	Converts device coordinates to a place name
POST /price, GET /retail	Retail (market vendor) pricing module from the first phase

4. PROJECT DESCRIPTION

4.1 Dataset Collection and Description

Unlike a project trained on a fixed historical file, the Adaptive Pricing Agent works on **live, per-trip data** assembled at request time from several sources, plus a **growing internal dataset** of the driver's own quotes and outcomes. Table 4 lists every source and the fields used.

Table 4: Data sources and fields used

Source	Collection method	Fields used
Driver input	Web form, validated by Pydantic	Pickup, drop, time, vehicle, competitor names and fares, taxi availability, fuel price, mileage, minimum profit, tolls, empty-return flag, language
OpenStreetMap Nominatim	HTTP search API (India only, ≤1 request/s)	Latitude, longitude, place name
Photon (komoot)	HTTP search and reverse API	Place suggestions with district and city context
OSRM	HTTP routing API	Driving distance (m), duration (s), route geometry
OpenWeather	Current-weather API by coordinates	Temperature, condition, description, rain intensity
Quote history (SQLite)	Written by the API after every quote	Route, recommended fare, rival anchor, demand pressure, outcome, final fare, time
Lookup cache (SQLite)	Written by the signal collectors	Geocodes, routes and searches, kept for 30 days

Table 5: Structure of a stored quote record

Field	Type	Description
id	TEXT	12-character unique quote identifier
created_at	REAL	Unix timestamp of the quote
pickup, drop_place	TEXT	Resolved place names
distance_km	REAL	Road distance from OSRM
recommended_fare	INTEGER	Fare recommended to the driver
anchor_name, anchor_fare	TEXT, REAL	Cheapest rival used as the market reference
demand_pressure	REAL	Demand-pressure score (0–1) at quote time
outcome	TEXT	'won', 'lost' or empty until the driver marks it
final_fare	INTEGER	Fare actually offered (may be a non-recommended option)

4.1.1 Data Validation and Cleaning

- Every request is validated by Pydantic: fares and costs must be positive, enumerations (vehicle, supply, weather, language) are restricted to known values, and malformed requests are rejected with a clear message.
- Competitor rows with an empty name or a non-positive fare are dropped before pricing.
- Place names are normalised (trimmed, lower-cased) before cache lookup; a suggestion picked in the UI carries coordinates so it is never geocoded again.
- If OSRM is unavailable, distance is estimated as the straight-line (haversine) distance × 1.35 and duration at 25 km/h; such estimates are not cached.
- If weather cannot be fetched, the trip is priced as dry and the reason is shown; driver-reported weather overrides the live reading.
- LLM output is parsed as JSON, its fare is clamped to the safe range, literal amounts are converted back into template placeholders, and Tamil messages must be at least 60% Tamil script.

4.2 Methodology

4.2.1 Cost Floor

The cost floor is the lowest fare at which the trip is still worth doing for the driver. With trip distance d (km), mileage m (km/L), fuel price f (₹/L), tolls and parking t and minimum profit p:

$$dead_km = d \text{ if the driver expects to return empty, else } 0$$

$$fuel_cost = (d + dead_km) / m \times f \qquad cost_floor = fuel_cost + t + p$$

For the reference trip (d = 11.1 km, m = 15 km/L, f = ₹102/L, p = ₹150) the fuel cost is ₹151 and the cost floor ₹301, which is charm-rounded up to ₹309.

4.2.2 Market Anchor

The market anchor is the cheapest competitor fare entered by the driver. When no competitor fare is entered, a typical regional fare is estimated as base + per_km × d + per_min × duration, multiplied by 1.2 when raining and 1.15 when taxis are scarce (branded operators surge in both cases):

Table 6: Market-estimate rates used when no rival fare is given

Vehicle	Base (₹)	Per km (₹)	Per minute (₹)
Hatchback	90	16	1.0
Sedan	100	19	1.0
SUV	150	25	1.5

4.2.3 Demand Pressure and Undercut

Demand pressure combines the signals that make riders accept a higher price. Each active signal adds a weight and the total is capped at 1:

Table 7: Demand-pressure weights

Signal	Condition	Weight
Rain	Light rain / heavy rain or thunderstorm	0.35 / 0.50
Taxi availability	Very few / some / plenty	0.40 / 0.15 / 0.00
Time band	Peak 07–10 h and 17–21 h / night 22–05 h	0.15 / 0.20

The undercut — how far below the market anchor the fare is set — falls linearly from 12% when demand is calm to 3% when it is maximal, because a scarce, rain-soaked market needs only a small discount to win the customer (Figure 4):

$$undercut = 0.12 \times (1 - pressure) + 0.03 \times pressure \qquad target = anchor \times (1 - undercut)$$

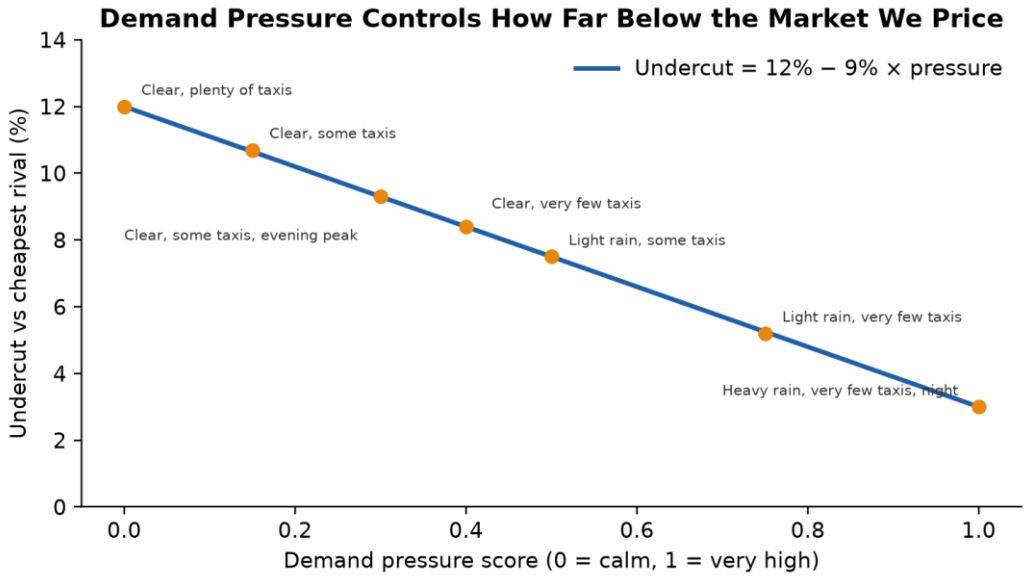


Figure 4: Undercut as a function of demand pressure, with the evaluated scenarios

4.2.4 Charm Pricing and Fare Options

The target is rounded down to the nearest fare ending in 9 (₹549.55 → ₹549); if that falls below the cost floor, the floor rounded up to a fare ending in 9 is used instead. Three options are produced: **quick win** (undercut + 4 points), **recommended** and **maximum earnings** (1.5% below the anchor). The safe range given to the LLM is [floor, largest fare ending in 9 below the anchor]; if the market is below the floor, the range collapses to the floor and no saving is claimed.

4.2.5 Traffic-Aware Travel Time

OSRM durations assume free-flowing roads. The displayed travel time is multiplied by 1.35 at peak hours, 1.15 at normal hours and 1.0 at night.

4.2.6 Learning From Outcomes

After at least five outcomes, the win rate w over the driver's last 20 marked quotes adjusts the undercut. A high win rate means money is being left on the table; a low win rate means the price is not attractive enough:

$$w \geq 0.75:\ adjustment = -\min(0.02, (w - 0.75) \times 0.08) \qquad w \leq 0.40:\ adjustment = +\min(0.03, (0.40 - w) \times 0.075)$$

$$undercut' = clamp(undercut + adjustment,\ 1\%,\ 15\%)$$

4.2.7 LLM Review and Guardrails

The LLM receives the trip, signals, rule-based fare, cost floor and safe range, and must answer with a JSON object containing a fare, headline, 3–4 reasons, a customer pitch and three selling points. The pitch is a template that uses {FARE}, {SAVINGS} and {COMPETITOR} placeholders, so the interface can fill it for whichever option the driver chooses. The response is rejected — and the deterministic template used instead — if it is not valid JSON, lacks the {FARE} placeholder, or is not mostly Tamil when Tamil was requested. Accepted fares are always clamped to the safe range, so the model can never produce a loss-making or uncompetitive price.

4.3 Implementation

4.3.1 MCP Tool Server

The signal collectors and the fare engine are exposed as typed MCP tools. One server process and session is opened per quote and reused for all tool calls:

```
mcp = FastMCP("taxi-signals")


@mcp.tool()
def get_route(pickup: str, drop: str, pickup_point: dict | None = None,
              drop_point: dict | None = None) -> dict:
    """Geocode pickup/drop and return driving distance, duration and the route path."""
    return route_collector.get_route(pickup, drop, pickup_point, drop_point)


@mcp.tool()
def compute_taxi_fare(distance_km: float, duration_min: float, competitor_fares: list[dict],
                      taxi_supply: str, is_raining: bool, ..., history: dict | None = None) ->
dict:
    return taxi_adaptors.compute_taxi_fare(...)
```

4.3.2 Fare Engine

```
dead_km = distance_km if return_empty else 0.0
fuel_cost = (distance_km + dead_km) / mileage_kmpl * fuel_price
cost_floor = fuel_cost + tolls_parking + min_profit
```

```
undercut = MAX_UNDERCUT * (1 - pressure) + MIN_UNDERCUT * pressure
adjustment = learned_adjustment(history)          # from booked / missed outcomes
if adjustment:
    undercut = min(max(undercut + adjustment, 0.01), 0.15)

floor_fare = _charm_up(cost_floor)
def priced(cut): return max(_charm_down(anchor_fare * (1 - cut)), floor_fare)
fare = priced(undercut)
```

4.3.3 LangGraph Orchestration

```
g = StateGraph(TaxiState)
g.add_edge(START, "load_history")                    # parallel branch 1
g.add_edge(START, "fetch_route")                     # parallel branch 2
g.add_edge("fetch_route", "fetch_weather")
g.add_edge(["load_history", "fetch_weather"], "price_trip")   # join
g.add_edge("price_trip", "call_llm")
g.add_conditional_edges("call_llm", route_after_llm,
                        {"fallback_advice": "fallback_advice", END: END})
g.add_edge("fallback_advice", END)
```

4.3.4 Streaming API

The streaming endpoint runs the compiled graph with LangGraph's update stream and forwards each finished node to the browser as one NDJSON line, so the fare appears as soon as the pricing node completes while the LLM is still writing:

```
async with _taxi_signals.connect() as signals:
    async for chunk in _taxi_graph.astream(state, config={'configurable': {'signals': signals}},
                                           stream_mode='updates'):
        for node, update in chunk.items():
            yield json.dumps({'type': 'node', 'node': node, 'data': update}) + '\n'
```

4.3.5 Web Application

The driver interface is a single HTML page with a dark, high-contrast theme designed for use in a car. It reads the NDJSON stream with the Fetch API, drives a live progress indicator from the real node events, animates the fare, shows rival comparison bars, a demand gauge, a Leaflet route map and a cost-versus-profit bar, and renders the customer message as a WhatsApp bubble with share and copy actions. Rain is animated on a canvas when it is raining. Driver costs, name, phone and language are remembered in the browser, and the layout adapts to phone screens.

4.3.6 Project Files

Table 8: Main source files

Module	Main files	Output
Fare engine	adaptors/taxi_adaptors.py	Fare, options, floor, factors, ETA
Signals	agents/route_collector.py, agents/signal_collector.py	Routes, places, weather
AI strategist	agents/taxi_strategist.py, agents/pricing_strategist.py	Prompt, validation, templates, Groq client
MCP	mcp_server/taxi_signals_server.py, mcp_server/client.py	Typed tools, persistent session

Module	Main files	Output
Orchestration	graph/taxi_state.py, taxi_nodes.py, taxi_build.py	Compiled LangGraph
Persistence	storage/store.py	SQLite history, stats and cache
API	server.py	REST and streaming endpoints
Frontend	web/taxi.html, web/retail.html	Driver and retail interfaces
Validation	tests/test_taxi_adaptors.py, tests/test_taxi_strategist.py	24 automated unit tests

4.3.7 Testing

Twenty-four unit tests cover charm pricing, beating the cheapest rival, the user's reference example, smaller undercuts under high demand, never pricing below the cost floor, the market estimate, empty-return cost, option ordering, traffic ETA, every branch of the learning rule, LLM parsing and clamping, placeholder conversion, malformed and wrong-language rejection, Tamil phrase repair, fallback templates and the SQLite store. The complete user flow — Tamil message, option switching and outcome recording — was also verified in headless Chrome at desktop and phone widths.

5. RESULTS AND INFERENCES

5.1 Reference Trip

The reference request is the example that motivated the project: a passenger at **Coimbatore Junction** wants to go to **Coimbatore International Airport**; Red Taxi quotes ₹600 and Blue Taxi ₹580; it is raining and there are very few taxis nearby. OSRM gives a 11.08 km road route of about 14 minutes. Table 9 shows the agent's quote.

Table 9: Quote for the reference trip

Quantity	Value
Cheapest rival (market anchor)	Blue Taxi, ₹580
Demand pressure	0.75 (rain 0.35 + very few taxis 0.40)
Undercut	5.2%
Fuel cost incl. empty return	₹151 for 22.2 km
Cost floor	₹301 → ₹309 after rounding
Recommended fare	**₹549** (₹31 below Blue Taxi)
Fare options	Quick win ₹519 · Recommended ₹549 · Max earnings ₹569
Driver keeps (after fuel)	₹398 at the recommended fare
Travel time with traffic	≈ 16 min

Inference: the agent wins the customer with a fare ₹31 cheaper than the cheapest brand while earning ₹398 — ₹248 more than the driver's minimum profit — because rain and scarcity mean a 5.2% discount is enough.

5.2 Scenario Analysis

The same trip was priced under seven combinations of weather, taxi availability and time (Table 10, Figure 5).

Table 10: Recommended fare under different demand conditions

Scenario	Pressure	Undercut	Fare	Saving vs ₹580	Driver keeps
Clear, plenty of taxis, afternoon	0.00	12%	₹509	₹71	₹358
Clear, some taxis, afternoon	0.15	10.7%	₹509	₹71	₹358
Clear, very few taxis, afternoon	0.40	8.4%	₹529	₹51	₹378
Clear, some taxis, evening peak	0.30	9.3%	₹519	₹61	₹368
Light rain, some taxis, afternoon	0.50	7.5%	₹529	₹51	₹378
Light rain, very few taxis, afternoon	0.75	5.2%	₹549	₹31	₹398

Scenario	Pressure	Undercut	Fare	Saving vs ₹580	Driver keeps
Heavy rain, very few taxis, night	1.00	3%	₹559	₹21	₹408

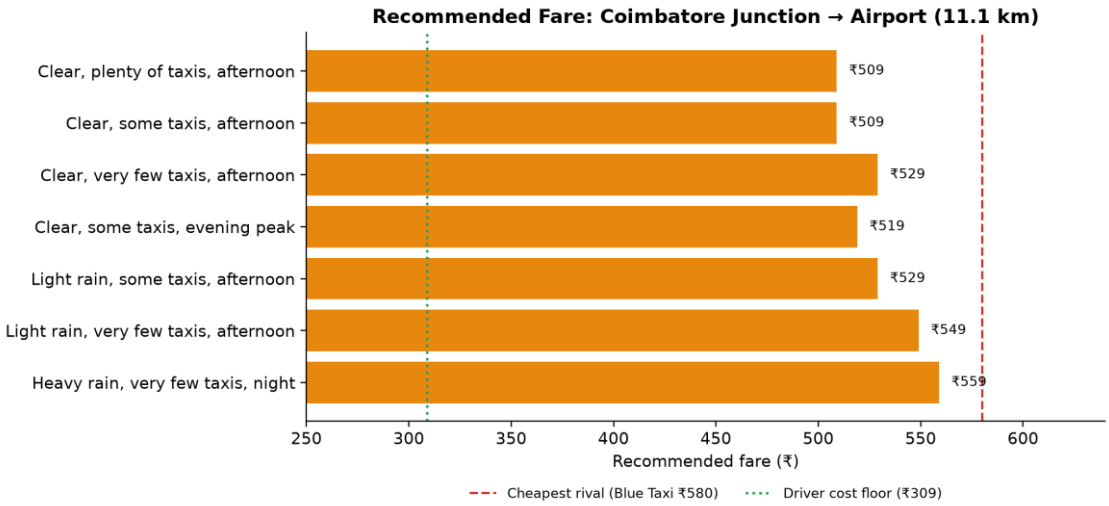


Figure 5: Recommended fare across demand scenarios versus the rival fare and cost floor

Inferences: (i) every recommendation is below the cheapest rival and above the cost floor; (ii) the fare rises monotonically with demand pressure, from ₹509 in calm conditions to ₹559 in heavy rain at night, so the driver earns ₹50 more exactly when riders have fewest alternatives; (iii) "plenty" and "some" taxis give the same ₹509 because both targets (₹510.4 and ₹517.9) round down to the same fare ending in 9 — charm rounding deliberately groups nearby targets.

5.3 Real Coimbatore Routes

Five real routes were priced with no rival quotes (market estimate), clear weather, some taxis and the empty return counted (Table 11, Figure 6).

Table 11: Results on real Coimbatore routes

Route	Km	Market est.	Floor	Fare	Driver keeps
Gandhipuram → Coimbatore International Airport	10.3	₹310	₹299	₹299	₹159
Ukkadam → Brookefields Mall	3.4	₹170	₹199	₹199	₹152
RS Puram → PSG College of Technology	7.7	₹254	₹259	₹259	₹154
Coimbatore Junction → Isha Yoga Information Center	31.6	₹731	₹589	₹649	₹220
Singanallur → Mettuppalayam	50.1	₹1,104	₹839	₹979	₹298

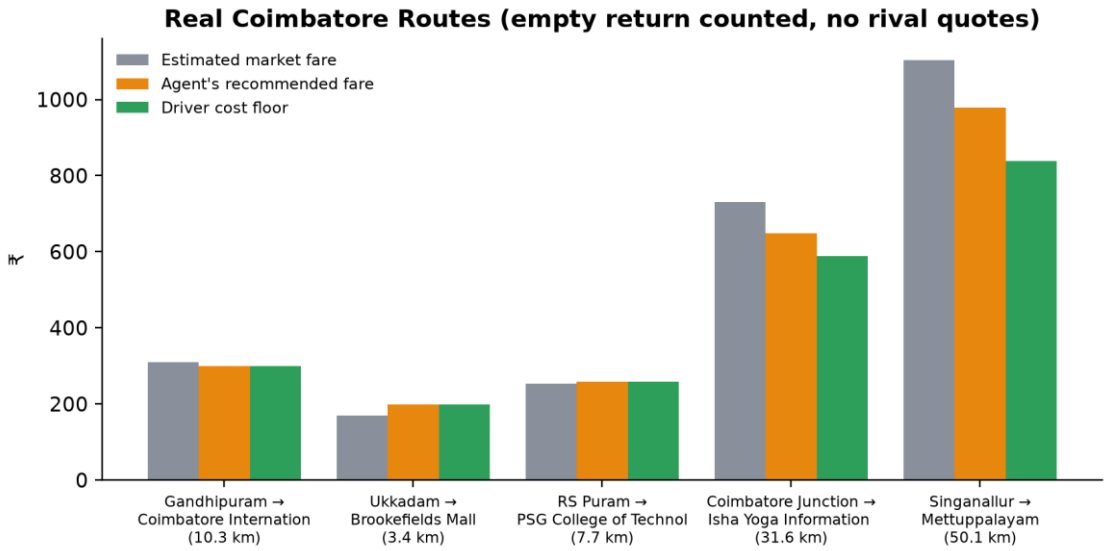


Figure 6: Market estimate, recommended fare and cost floor on real routes

Inferences: on long trips (31.6 km and 50.1 km) the agent undercuts the market by ₹82 and ₹125 while keeping ₹220–₹298. On short city trips the cost floor — dominated by the minimum profit and the empty return — is at or above the market estimate, so the agent quotes the floor and does not claim savings. This tells the driver that short trips are only competitive if they do not count an empty return (for example when another passenger is likely nearby) or accept a smaller minimum profit.

5.4 Learning From Outcomes

Table 12 and Figure 7 show the effect of the driver's win rate over ten marked quotes on the reference trip under calm conditions.

Table 12: Effect of the driver's win rate on the recommendation

Win rate	Undercut adjustment	Fare
0%	+3 points	₹499
20%	+1.5 points	₹509
40%	0 points	₹509
60%	0 points	₹509
80%	-0.4 points	₹519
90%	-1.2 points	₹519
100%	-2 points	₹529

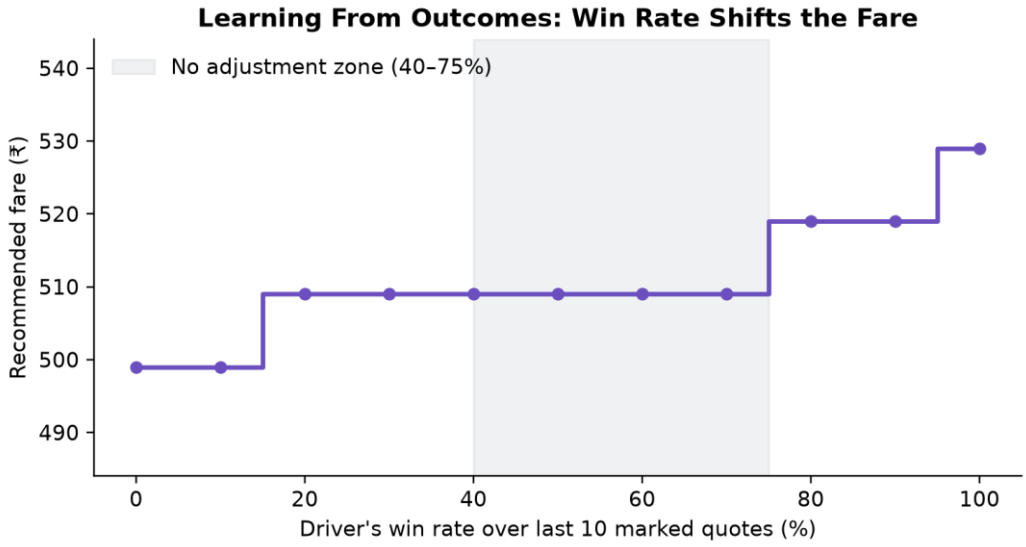


Figure 7: Recommended fare as a function of the driver's win rate

Inference: a driver who wins every quote is automatically moved from ₹509 to ₹529 (+₹20 per trip), while a driver who loses most quotes is moved down to ₹499. Win rates between 40% and 75% leave the price unchanged, which avoids over-reacting to noise.

5.5 AI Reliability Evaluation

The LLM path was evaluated with 20 live calls (10 English, 10 Tamil) using the production prompt and validator on the reference trip (Table 13, Figure 8).

Table 13: LLM guardrail evaluation (n = 20)

Metric	English	Tamil
Valid JSON returned	10/10	10/10
Accepted by validator	10/10	9/10
Rejected → template used	0/10	1/10 (mostly not Tamil)
Accepted fares inside safe range	10/10	9/9
Mean response time	2.18 s	2.62 s

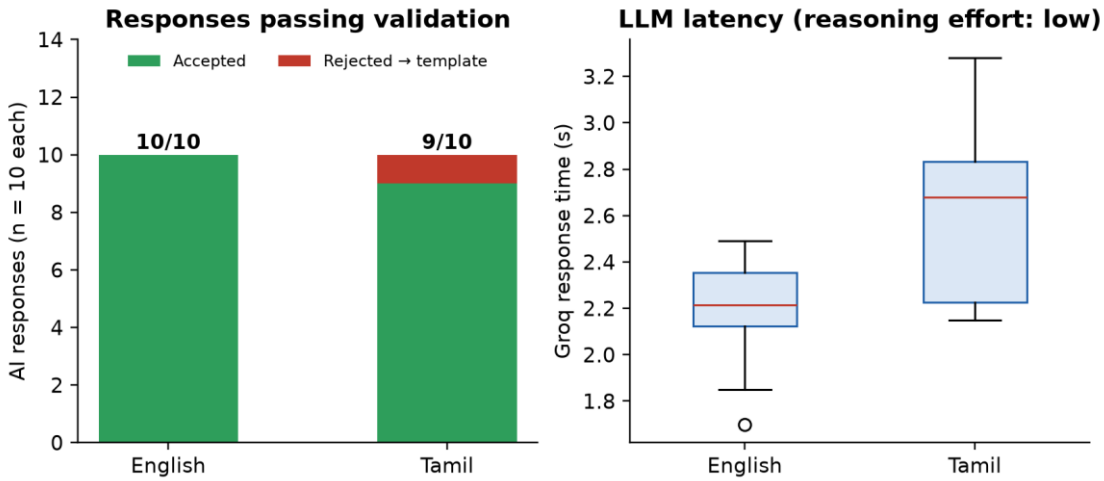


Figure 8: LLM responses passing validation and response-time distribution

Inference: the model followed the template and language instructions in 95% of calls; the remaining call was caught by the language check and replaced by the Tamil template, so the driver still received a correct Tamil message. In every accepted response the model kept the rule-based fare, confirming that the deterministic engine — not the LLM — determines the price.

5.6 Performance

The first working version took 18.7–23.0 seconds per quote because each of three tool calls started a new MCP server process, map lookups were repeated and the LLM used its default reasoning effort. One persistent MCP session per quote, SQLite caching, low reasoning effort and streaming reduced this substantially (Table 14, Figure 9).

Table 14: Measured end-to-end latency

Configuration	Time
Before optimisation — complete quote (mean of 2 runs)	20.8 s
After optimisation — complete quote (mean of 5 runs)	6.0 s
After optimisation — fare visible to the driver	3.4 s

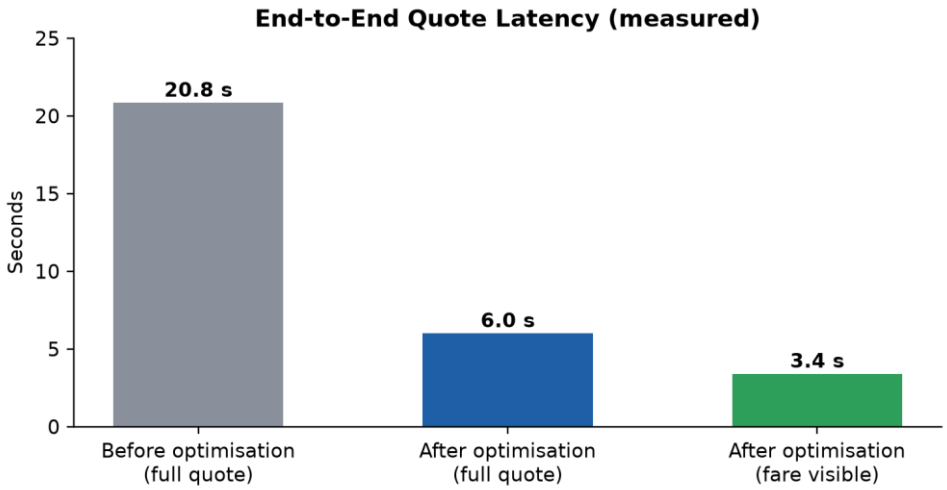


Figure 9: End-to-end latency before and after optimisation

5.7 Output Evidence

The following screenshots were captured from the running application.

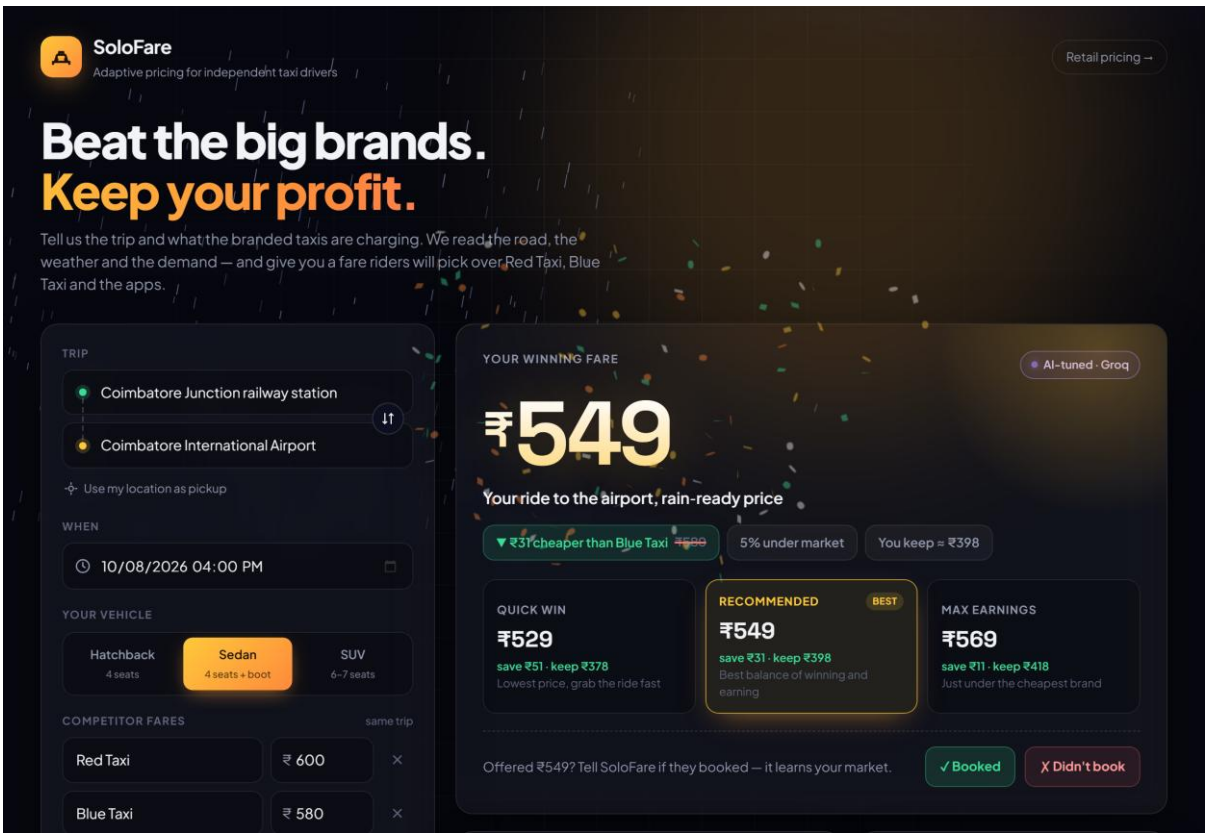


Figure 10: Application home screen with trip form and the winning fare

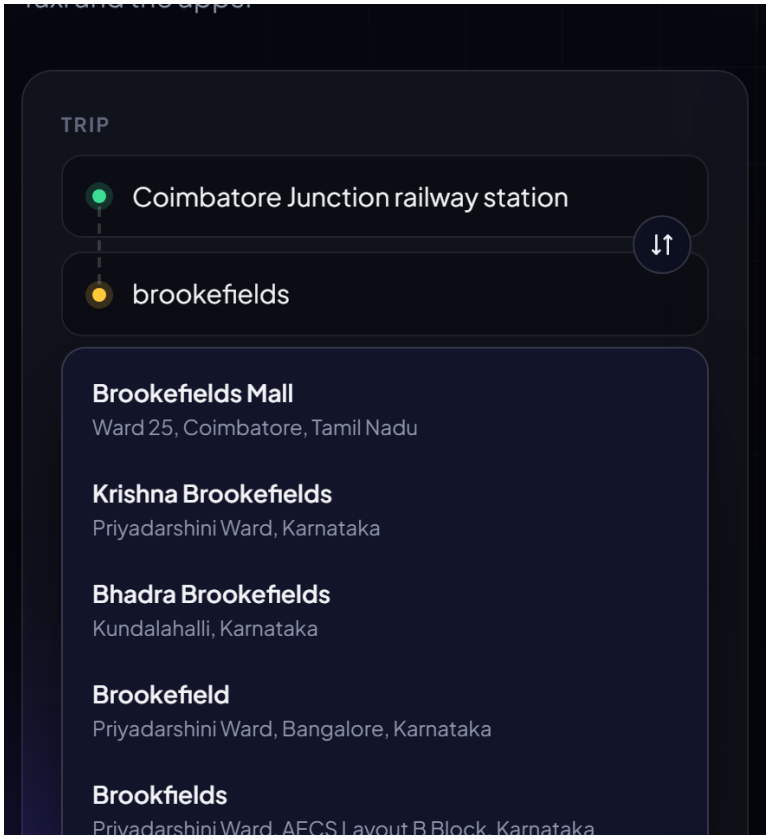


Figure 11: Search-as-you-type place suggestions biased to Coimbatore

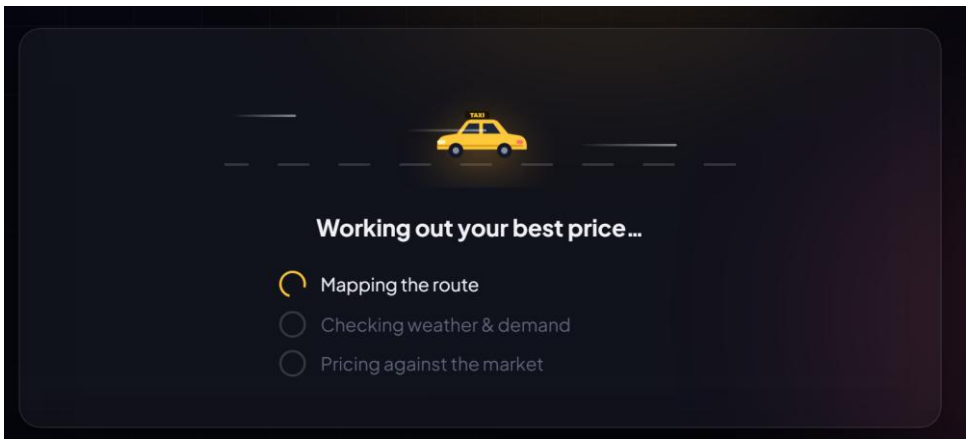


Figure 12: Live progress driven by the streamed graph nodes

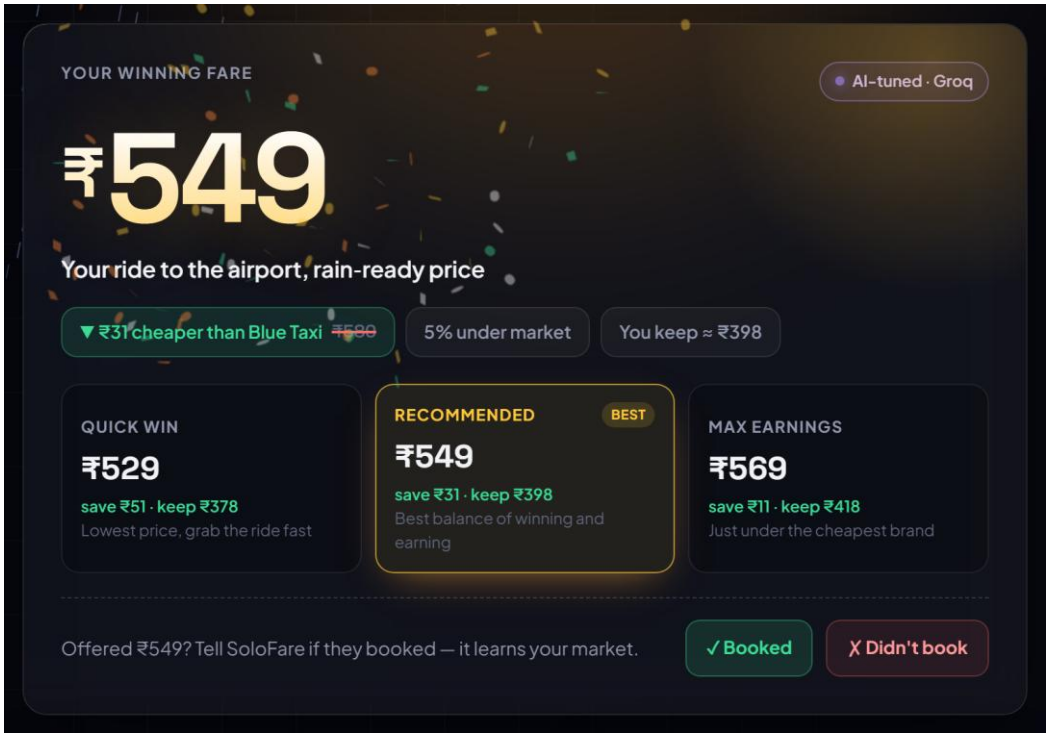


Figure 13: Winning fare card with three options and outcome buttons



Figure 14: Comparison with rival fares and the demand-pressure gauge

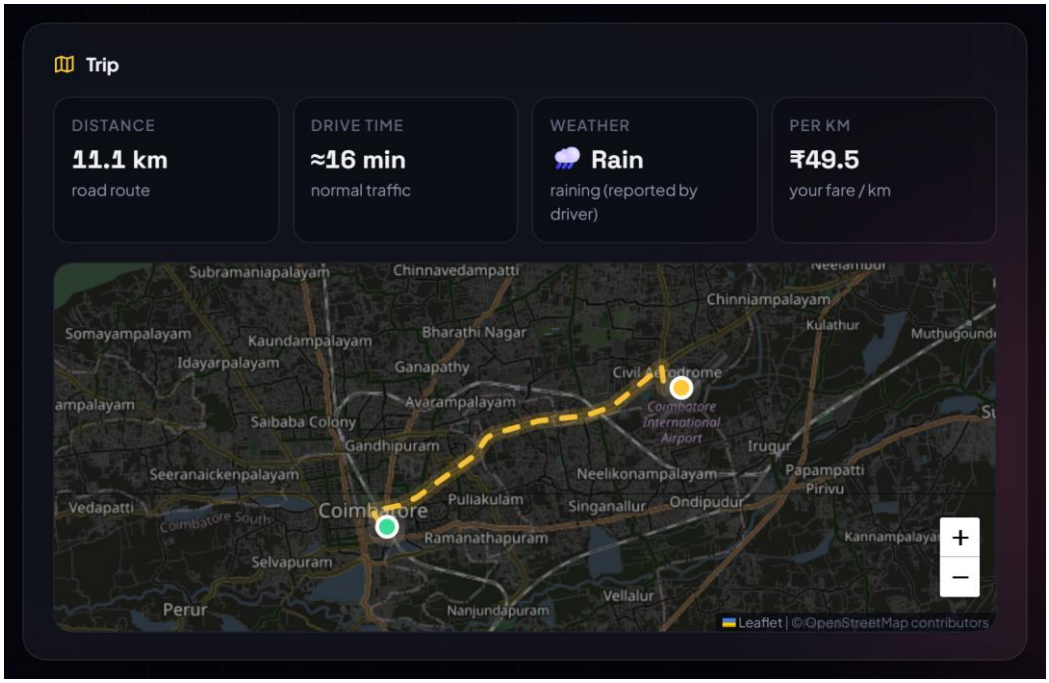


Figure 15: Trip details and route map

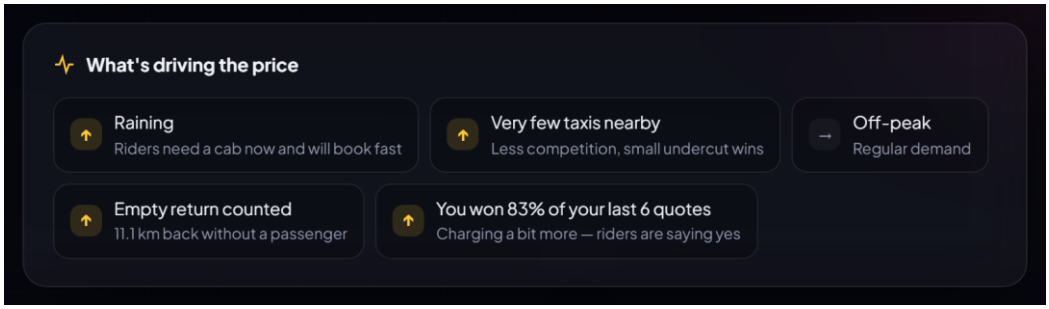


Figure 16: Factors driving the price, including the learned win-rate factor

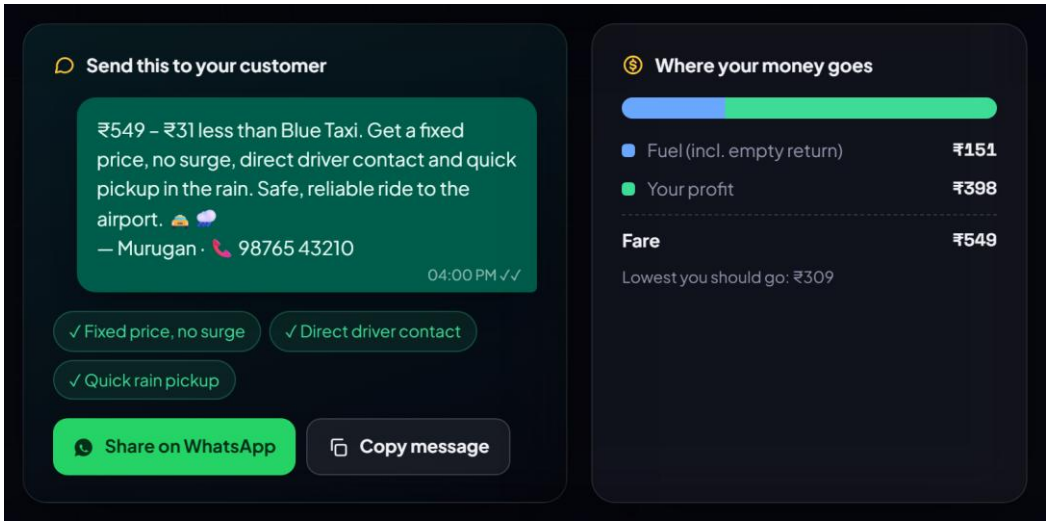


Figure 17: WhatsApp-ready customer message and money breakdown



Figure 18: AI explanation of the recommended price

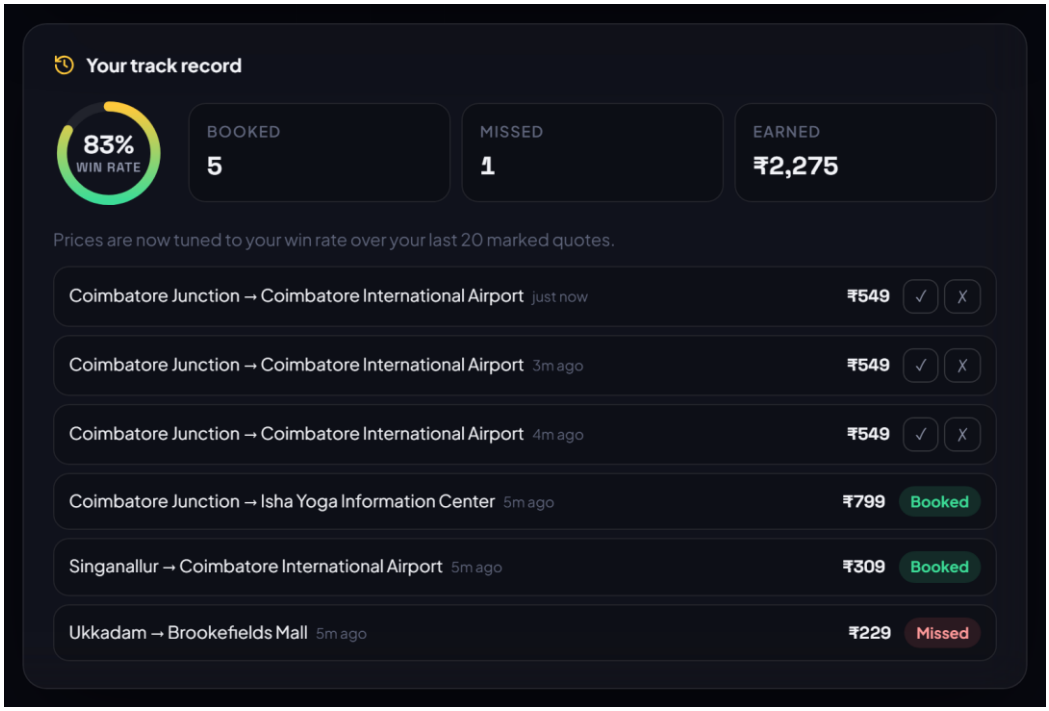


Figure 19: Driver track record: win rate, earnings and recent quotes

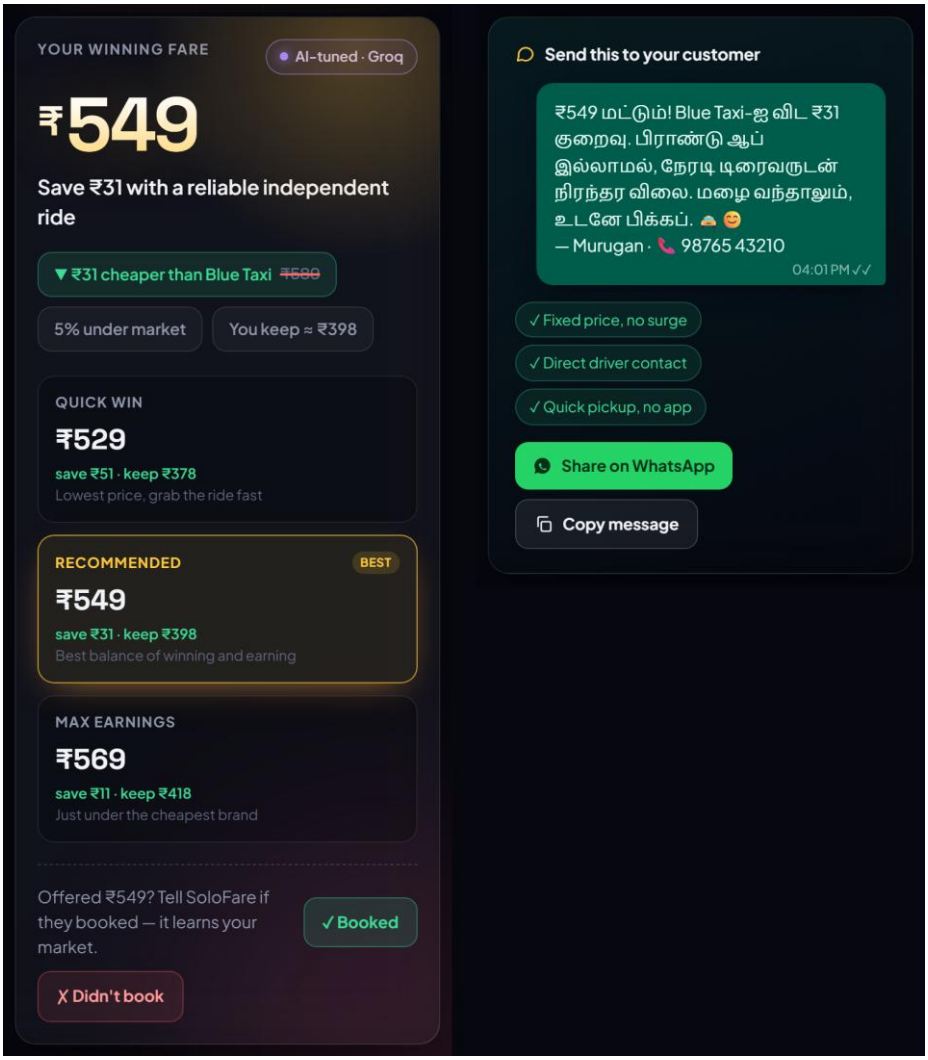


Figure 20: Phone layout with a Tamil customer message

```
$ curl -N -X POST http://localhost:8000/taxi/quote/stream  (one JSON line per finished graph node)

{"type": "node", "node": "load_history", "data": {"history": {"decided": 0, "won": 0, "lost": 0, "win_rate": null, "ea
{"type": "node", "node": "fetch_route", "data": {"route": {"pickup": {"lat": 10.9975682, "lon": 76.9663657, "name": "C
{"type": "node", "node": "fetch_weather", "data": {"weather": {"temp_c": null, "condition": "Rain", "description": "ra
{"type": "node", "node": "price_trip", "data": {"quote": {"fare": 549, "options": [{"key": "quick", "label": "Quick wi
{"type": "node", "node": "call_llm", "data": {"advice": {"fare": 549, "headline": "Ride cheap, rain\u2011ready, no sur
{"type": "done", "quote_id": "d3e9a357ae33", "result": {"route": {"pickup": {"lat": 10.9975682, "lon": 76.9663657, "na
```

Figure 21: Streaming API output — one JSON line per completed graph node

```
$ python -m unittest discover tests -v

test_charm_down (adaptors.CharmPricing.test_charm_down)  ... ok
test_charm_up (adaptors.CharmPricing.test_charm_up)  ... ok
test_losing_streak_prices_sharper (adaptors.Learning.test_losing_streak_prices_sharper)  ... ok
test_middle_win_rate_changes_nothing (adaptors.Learning.test_middle_win_rate_changes_nothing)  ... ok
test_needs_five_outcomes (adaptors.Learning.test_needs_five_outcomes)  ... ok
test_winning_streak_charges_more (adaptors.Learning.test_winning_streak_charges_more)  ... ok
test_beats_cheapest_competitor (adaptors.TaxiFare.test_beats_cheapest_competitor)  ... ok
test_empty_return_raises_cost (adaptors.TaxiFare.test_empty_return_raises_cost)  ... ok
test_eta_includes_traffic_at_peak (adaptors.TaxiFare.test_eta_includes_traffic_at_peak)  ... ok
test_high_demand_means_smaller_undercut (adaptors.TaxiFare.test_high_demand_means_smaller_undercut)  ... ok
test_never_below_cost_floor (adaptors.TaxiFare.test_never_below_cost_floor)  ... ok
test_no_quotes_uses_market_estimate (adaptors.TaxiFare.test_no_quotes_uses_market_estimate)  ... ok
test_options_are_ordered_and_profitable (adaptors.TaxiFare.test_options_are_ordered_and_profitable)  ... ok
test_user_example_rain_and_few_taxis (adaptors.TaxiFare.test_user_example_rain_and_few_taxis)  ... ok
test_templates_have_placeholders_in_both_languages (strategist.FallbackAdvice.test_templates_have_placeholders_in_both_languages)  ... ok
test_english_pitch_rejected_when_tamil_requested (strategist.ParseAdvice.test_english_pitch_rejected_when_tamil_requested)  ... ok
test_english_savings_phrase_is_translated_in_tamil (strategist.ParseAdvice.test_english_savings_phrase_is_translated_in_tamil)  ... ok
test_fare_is_clamped_into_allowed_range (strategist.ParseAdvice.test_fare_is_clamped_into_allowed_range)  ... ok
test_literal_numbers_are_turned_into_placeholders (strategist.ParseAdvice.test_literal_numbers_are_turned_into_placeholders)  ... ok
test_malformed_json_is_rejected (strategist.ParseAdvice.test_malformed_json_is_rejected)  ... ok
test_pitch_without_fare_is_rejected (strategist.ParseAdvice.test_pitch_without_fare_is_rejected)  ... ok
test_valid_template_passes (strategist.ParseAdvice.test_valid_template_passes)  ... ok
test_cache_roundtrip (strategist.Store.test_cache_roundtrip)  ... ok
test_outcomes_feed_stats (strategist.Store.test_outcomes_feed_stats)  ... ok
----------------------------------------------------------------------
Ran 24 tests in 0.106s
OK
```

Figure 22: Automated test suite output — all 24 tests passed

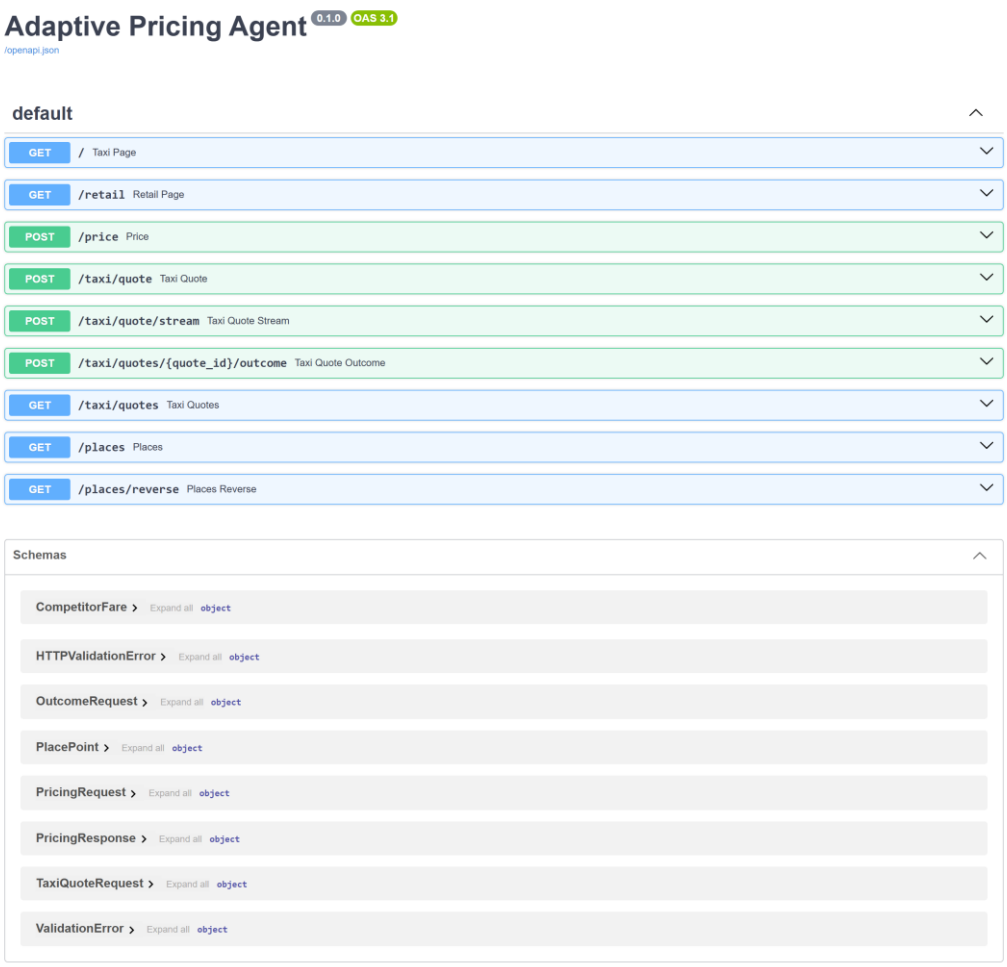


Figure 23: Interactive API documentation generated by FastAPI

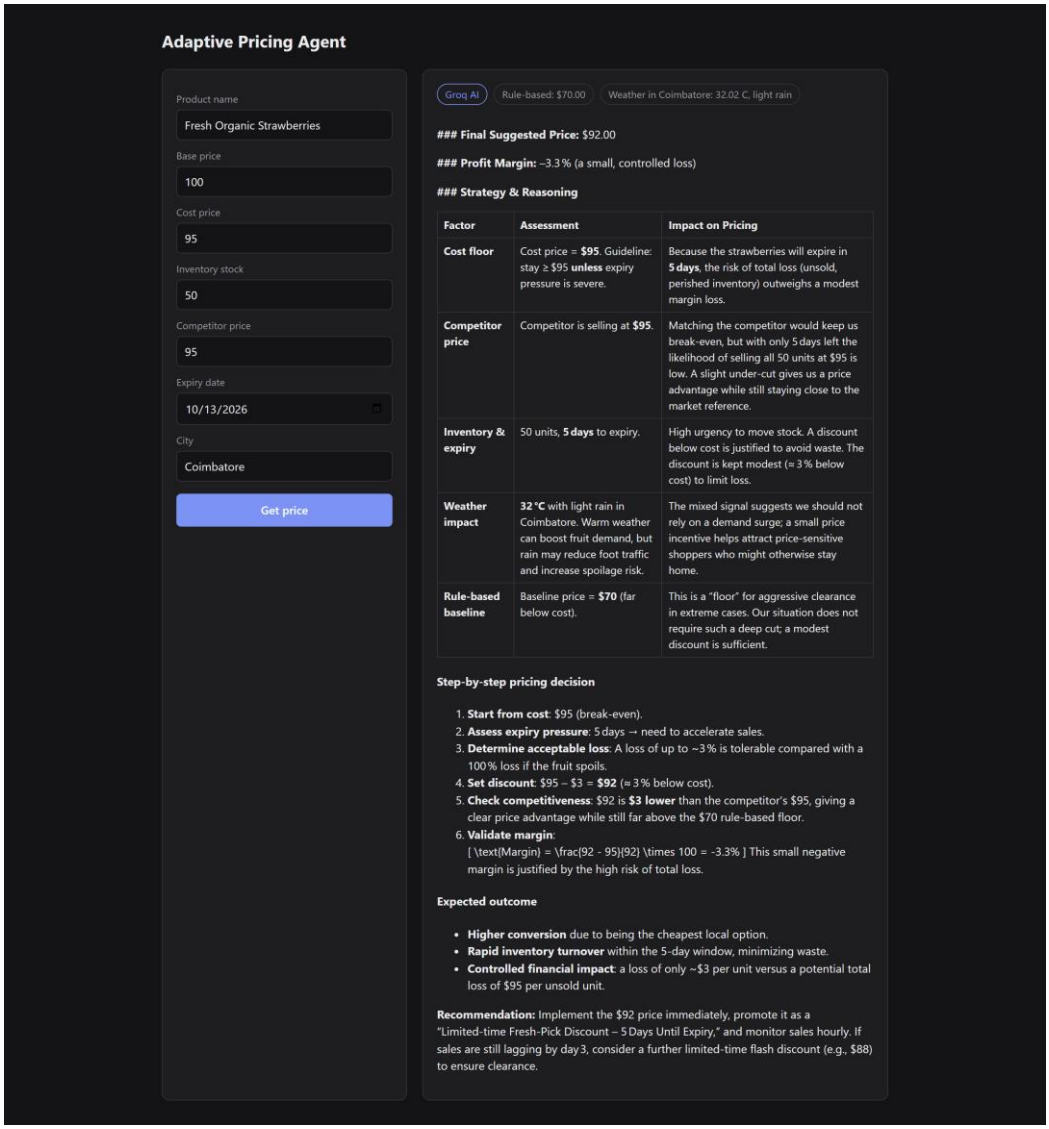


Figure 24: Retail pricing module from the first project phase

6. CONCLUSION & FUTURE SCOPE

6.1 Conclusion

The Adaptive Pricing Agent gives an independent taxi driver the kind of pricing intelligence that branded fleets and ride-hailing platforms already have. For every trip it collects live route, weather, time and competition signals, computes the driver's true cost including the empty return, measures demand pressure and recommends a fare that beats the cheapest branded rival whenever that is profitable. On the motivating example — Coimbatore Junction to the airport in the rain with few taxis around and rivals at ₹600 and ₹580 — it recommends ₹549, ₹31 cheaper than the cheapest brand while the driver keeps ₹398.

The project combines a transparent, testable pricing model with a modern agent architecture: tools exposed through the Model Context Protocol, a LangGraph state graph with parallel and fallback paths, a guard-railed LLM that explains the price and writes a bilingual customer message, a learning loop driven by real booking outcomes, and a streaming web interface. The evaluation shows that recommendations are always above the cost floor and below the rival when possible, that higher demand earns the driver more, that 95% of LLM responses pass validation with the remainder safely replaced, and that a quote is delivered in about six seconds with the fare visible after 3.4 seconds.

The main limitations are that taxi availability is reported by the driver rather than measured, competitor fares must be entered manually, the market-estimate rates are fixed regional assumptions, OSRM durations do not include live traffic, and the learning rule uses a single win rate for all routes. Fare recommendations are decision support; the driver remains responsible for the final price.

6.2 Future Scope

- Learn separate win rates per route type (airport, railway station, short city hop, outstation) instead of one global rate.
- Fit a demand (acceptance-probability) model from accumulated outcomes, for example logistic regression on fare gap, pressure and time, and choose the fare that maximises expected profit.
- Estimate taxi scarcity automatically from time, location and event calendars, and obtain live traffic from a traffic-aware routing service.
- Add voice input and output in Tamil so the driver can use the agent hands-free.
- Support shared rides, waiting charges, multiple stops and night surcharges.

- Package the interface as an installable progressive web app with offline caching of recent routes.
- Extend the same agent design to other independent services — goods carriers, bike taxis and home-stay hosts — as separate agents that communicate through the Agent-to-Agent (A2A) protocol.
- Add observability (LangSmith or Langfuse) and container-based deployment for multi-driver use.

6.3 Final Inference

The central contribution of the project is a safe division of labour between a deterministic pricing engine and a language model: the engine guarantees that every price is profitable and explainable, while the model turns that price into words a customer responds to. This makes the system a practical, data-driven decision-support tool for independent drivers and a working demonstration of how agentic AI can be applied responsibly to a real pricing problem.

REFERENCES

[1] G. Gallego and G. van Ryzin, “Optimal dynamic pricing of inventories with stochastic demand over finite horizons,” Management Science, vol. 40, no. 8, pp. 999–1020, 1994.

[2] A. V. den Boer, “Dynamic pricing and learning: Historical origins, current research, and new directions,” Surveys in Operations Research and Management Science, vol. 20, no. 1, pp. 1–18, 2015.

[3] G. P. Cachon, K. M. Daniels and R. Lobel, “The role of surge pricing on a service platform with self-scheduling capacity,” Manufacturing & Service Operations Management, vol. 19, no. 3, pp. 368–384, 2017.

[4] J. C. Castillo, D. Knoepfle and G. Weyl, “Surge pricing solves the wild goose chase,” in Proc. ACM Conference on Economics and Computation (EC ’17), pp. 241–242, 2017.

[5] L. Chen, A. Mislove and C. Wilson, “Peeking beneath the hood of Uber,” in Proc. ACM Internet Measurement Conference (IMC ’15), pp. 495–508, 2015.

[6] P. Cohen, R. Hahn, J. Hall, S. Levitt and R. Metcalfe, “Using big data to estimate consumer surplus: The case of Uber,” NBER Working Paper No. 22627, 2016.

[7] M. Thomas and V. Morwitz, “Penny wise and pound foolish: The left-digit effect in price cognition,” Journal of Consumer Research, vol. 32, no. 1, pp. 54–64, 2005.

[8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in Proc. International Conference on Learning Representations (ICLR), 2023.

[9] Anthropic, “Model Context Protocol specification,” modelcontextprotocol.io, 2024.

[10] LangChain Inc., “LangGraph documentation,” langchain-ai.github.io/langgraph.

[11] M. Haklay and P. Weber, “OpenStreetMap: User-generated street maps,” IEEE Pervasive Computing, vol. 7, no. 4, pp. 12–18, 2008.

[12] D. Luxen and C. Vetter, “Real-time routing with OpenStreetMap data,” in Proc. 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, pp. 513–516, 2011.

[13] OpenWeather Ltd., “Current weather data API documentation,” openweathermap.org/current.

[14] S. Ramírez, “FastAPI documentation,” fastapi.tiangolo.com.